



Chuyên đề: Kỹ thuật Rời rạc hóa & Nén Tọa độ (Coordinate Compression & Discretization)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

🎯 1. Tổng quan & Bản đồ Tư duy (Mindmap)

Trong lập trình thi đấu (Competitive Programming), chúng ta rất thường xuyên gặp các bài toán có đặc trưng:

- Không gian giá trị các phần tử hoặc tọa độ cực kỳ lớn: $A_i, X_i, Y_i \in [-10^9, 10^9]$ hoặc $[-10^{18}, 10^{18}]$.
- Tuy nhiên, số lượng phần tử hoặc sự kiện thực tế lại rất nhỏ: $N \leq 10^5$ hoặc $2 \cdot 10^5$.
- Thuật toán giải quyết bài toán lại yêu cầu đánh chỉ số mảng (Direct Addressing), ví dụ:
- Mảng đếm tần số `cnt[x]`.
- Cây chỉ số nhị phân (Fenwick Tree / BIT) hoặc Cây phân đoạn (Segment Tree) kích thước $O(N)$.
- Mảng hiệu (Difference Array) và Quét đường (Line Sweep).
- Ma trận lưới 2D (`grid[r][c]`) để loang Flood Fill BFS/DFS.

Nếu cố tình khai báo mảng với chỉ số 10^9 , chương trình sẽ bị **MLE (Memory Limit Exceeded)** hoặc **CE (Compile Error)** ngay lập tức.

Kỹ thuật Rời rạc hóa (Coordinate Compression / Discretization) là phương pháp ánh xạ (mapping) toàn bộ không gian giá trị khổng lồ và thừa thớt đó về một tập các số nguyên liên tiếp nhỏ $\{0, 1, 2, \dots, K - 1\}$ ($K \leq N$ hoặc $2N$) sao cho **thứ tự tương đối giữa các phần tử vẫn được bảo toàn tuyệt đối**:

$$x < y \iff \text{rank}(x) < \text{rank}(y)$$



Syntax error in text
mermaid version 10.9.8

2. Nén Điểm 1D (Point Compression)

2.1. C++ Idiom Chuẩn mực: `sort` + `unique` + `erase`

Giả sử ta có mảng A gồm N số nguyên lớn:

$$A = [10^9, -5 \cdot 10^8, 10^9, 42, 42, -5 \cdot 10^8]$$

Quy trình nén gồm đúng 4 bước chuẩn mực:

1. **Tạo mảng bản sao:** `vector<long long> vals = A;`

2. **Sắp xếp tăng dần:** `std::sort(vals.begin(), vals.end());`

$$\Rightarrow vals = [-5 \cdot 10^8, -5 \cdot 10^8, 42, 42, 10^9, 10^9]$$

3. **Loại bỏ phần tử trùng lặp:**

`cpp vals.erase(std::unique(vals.begin(), vals.end()), vals.end());`

$$\Rightarrow vals = [-5 \cdot 10^8, 42, 10^9] \text{ với kích thước } K = 3.$$

4. **Ánh xạ sang chỉ số nén (0-based hoặc 1-based):**

Với mỗi giá trị $x \in A$, thứ hạng nén của x được tìm bằng tìm kiếm nhị phân `std::lower_bound`:

`cpp int rank = lower_bound(vals.begin(), vals.end(), x) - vals.begin();`

- Giá trị $-5 \cdot 10^8 \rightarrow \text{rank} = 0$.

- Giá trị $42 \rightarrow \text{rank} = 1$.

- Giá trị $10^9 \rightarrow \text{rank} = 2$.

Mảng A ban đầu được biến đổi thành mảng nén:

$$A_{\text{compressed}} = [2, 0, 2, 1, 1, 0]$$

2.2. Phục hồi Giá trị Gốc (Decompression)

Để lấy lại giá trị thực ban đầu từ chỉ số nén `rank`, ta chỉ cần truy cập trực tiếp phần tử trong mảng `vals` trong $O(1)$:

$$\text{original_val} = \text{vals}[\text{rank}]$$

Phân tích Độ phức tạp:

- Thời gian tiền xử lý: $O(N \log N)$ cho bước sắp xếp.

- Bộ nhớ phụ: $O(N)$ lưu mảng `vals`.

- Thời gian ánh xạ mỗi phần tử: $O(\log N)$ với `lower_bound`, hoặc $O(1)$ nếu tiền xử lý ánh xạ trực tiếp vào mảng.

3. Nén Đoạn 1D & Cạm bẫy Khoảng hở (Interval Compression & Gap Pitfall)

3.1. Cạm bẫy Khoảng hở (The Missing Gap Pitfall)

Xét bài toán: "Cho N đoạn thẳng đóng $[L_i, R_i]$ trên trục số. Hãy đếm số điểm nguyên được phủ bởi ít nhất một đoạn thẳng".

Giả sử có 2 đoạn: $[1, 2]$ và $[4, 5]$.

- Tập hợp các đầu mút: $\{1, 2, 4, 5\}$.

- Nếu ta nén thông thường:

- $1 \rightarrow 0$

- $2 \rightarrow 1$

- $4 \rightarrow 2$

- $5 \rightarrow 3$

- Đoạn $[1, 2]$ trở thành $[0, 1]$. Đoạn $[4, 5]$ trở thành $[2, 3]$.

- Quan sát thấy: Điểm 2 (nén thành 1) và điểm 4 (nén thành 2) bỗng nhiên trở thành **hai điểm kề sát nhau** trong không gian nén!

- Khoảng hở (Gap) giữa 2 và 4 (chính là điểm 3 không bị đoạn nào phủ) đã **hoàn toàn biến mất!**

- Nếu bài toán yêu cầu tìm số thành phần liên thông của các đoạn phủ, thuật toán nén ngây thơ sẽ cho rằng hai đoạn phủ tách biệt lại vô tình chạm sát nhau, dẫn đến kết quả sai nghiêm trọng!

3.2. Kỹ thuật Chèn Điểm Trung gian (Intermediate Points Insertion)

Để bảo toàn các khoảng hở tự nhiên giữa các đoạn rời nhau, nguyên tắc vàng là:

 **Nguyên tắc:** Với mỗi mốc tọa độ $x \in \{L_i, R_i\}$, ta chèn thêm cả $x - 1$ (hoặc cả $x - 1$ và $x + 1$) vào tập giá trị cần nén!

Quay lại ví dụ trên:

- Các điểm đầu mút: $\{1, 2, 4, 5\}$.

- Thêm các điểm liền kề: $\{1, 1 + 1 = 2, 2 + 1 = 3, 4, 4 + 1 = 5\}$.

- Tập hợp điểm đầy đủ: $\{1, 2, 3, 4, 5\}$.

- Khi nén:

- $1 \rightarrow 0$

- $2 \rightarrow 1$

- $3 \rightarrow 2$ (Đại diện cho khoảng hở giữa 2 và 4!)

- $4 \rightarrow 3$

- $5 \rightarrow 4$

- Bây giờ, đoạn $[1, 2]$ là $[0, 1]$, đoạn $[4, 5]$ là $[3, 4]$. Điểm nén 2 (tương ứng với giá trị thực 3) nằm trơ trọi ở giữa và không bị phủ bởi đoạn nào. Cấu trúc topo của không gian được bảo toàn 100%!

3.3. Tính Độ dài Đoạn Nén (Weighted Segment Intervals)

Khi nén các đoạn liên tục trên trục thực để tính diện tích hoặc chiều dài hợp đoạn:

- Mỗi chỉ số nén i không chỉ là một điểm đơn lẻ, mà đại diện cho đoạn nửa mở:

$$I_i = [vals[i], vals[i + 1])$$

- Độ dài thực tế của khoảng nén thứ i là:

$$\text{length}(i) = \text{vals}[i + 1] - \text{vals}[i]$$

- Khi đó, thuật toán Mảng hiệu (Difference Array) hoặc Segment Tree trên các khoảng nén sẽ cộng dồn trọng số $\text{length}(i)$ thay vì chỉ cộng 1.

4. Rời rạc hóa Lưới 2D (2D Grid Coordinate Compression)

4.1. Bài toán Lưới Siêu Lớn với Chương ngại vật Thừa

Xét bài toán: Một mê cung kích thước $10^9 \times 10^9$. Có $N \leq 1000$ bức tường hình chữ nhật $[x_1, x_2] \times [y_1, y_2]$. Cần kiểm tra xem có đường đi từ (S_x, S_y) tới (T_x, T_y) mà không chạm tường hay không.

- Kích thước $10^9 \times 10^9$ khiến ta không thể tạo mảng 2D.

- Tuy nhiên, $N \leq 1000$ bức tường chỉ tạo ra tối đa 2000 mốc tọa độ X và 2000 mốc tọa độ Y !

4.2. Quy trình Nén Lưới 2 Chiều Độc lập

1. **Thu thập tọa độ:**

2. Trục X : Gom toàn bộ $\{x_1, x_2, x_1 - 1, x_2 + 1, S_x, T_x\}$.

3. Trục Y : Gom toàn bộ $\{y_1, y_2, y_1 - 1, y_2 + 1, S_y, T_y\}$.

4. **Nén độc lập:**

5. Nén tập X thành vals_X kích thước $W \leq 4000$.

6. Nén tập Y thành vals_Y kích thước $H \leq 4000$.

7. **Dựng ma trận nén:**

8. Khởi tạo ma trận `grid[W][H]` có kích thước tối đa $\approx 4000 \times 4000$ (chỉ tốn khoảng 16 MB bộ nhớ, hoàn toàn an toàn).

9. Mỗi bức tường $[x_1, x_2] \times [y_1, y_2]$ được ánh xạ thành hình chữ nhật con $[\text{rank}_X(x_1), \text{rank}_X(x_2)] \times [\text{rank}_Y(y_1), \text{rank}_Y(y_2)]$ trên lưới mới.

10. Dùng **2D Difference Array** để đánh dấu các ô tường trong $O(N)$ hoặc đánh dấu trực tiếp.

11. **Chạy BFS/DFS:**

12. Chạy Flood Fill BFS xuất phát từ $(\text{rank}_X(S_x), \text{rank}_Y(S_y))$ tới $(\text{rank}_X(T_x), \text{rank}_Y(T_y))$ trên ma trận nén trong $O(W \cdot H) \approx O(N^2)$ thời gian!

5. Rời rạc hóa Động (Online Discretization)

Khi bài toán xuất hiện các giá trị trực tuyến (Online) mà không cho trước toàn bộ mảng ban đầu:

5.1. Bằng `std::map<long long, int>`

```
struct OnlineCompressor {
    map<long long, int> rank_map;
    vector<long long> orig_vals;

    int get_rank(long long x) {
        auto it = rank_map.find(x);
        if (it != rank_map.end()) return it->second;
        int new_rank = orig_vals.size();
        rank_map[x] = new_rank;
        orig_vals.push_back(x);
        return new_rank;
    }
};
```

- **Ưu điểm:** Nạp phần tử bất kỳ lúc nào, không cần biết trước N .
- **Nhược điểm:** Không bảo toàn thứ tự nếu chen xáo trộn (chỉ dùng để bấm định danh), hoặc nếu dùng `std::map` để duyệt theo thứ tự thì mất $O(\log N)$ mỗi thao tác.

5.2. Bằng GNU C++ PBDS (Policy Based Data Structure)

Để vừa thêm phần tử online vừa biết chính xác thứ tự của nó trong tập hợp hiện tại, ta sử dụng **Order Statistic Tree** từ thư viện nội trang GCC:

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

typedef tree<long long, null_type, less<long long>, rb_tree_tag, tree_order_statistics_node_update>
ordered_set os;
os.insert(100);
os.insert(42);
os.insert(500);

// os.order_of_key(x): Trả về số lượng phần tử trong tập hợp nhỏ hơn ngặt x (chính là thứ hạng của x)
int rank_42 = os.order_of_key(42); // 0 (nhỏ nhất)
int rank_100 = os.order_of_key(100); // 1
int rank_500 = os.order_of_key(500); // 2
```

- Hỗ trợ cả `insert`, `erase` và truy vấn `order_of_key` (thứ hạng nén) trong $O(\log N)$ trực tuyến!

6. Bảng So sánh Các Kỹ thuật Rời rạc hóa

Kỹ thuật	Đối tượng áp dụng	Ưu điểm	Hạn chế / Cạm bẫy	Độ phức tạp
Nén Điểm 1D	Dãy số, mảng phần tử	Cực kỳ ngắn gọn, bộ nhớ liên tục	Chỉ bảo toàn thứ tự điểm, không bảo toàn độ dài	$O(N \log N)$
Nén Đoạn 1D (+ Gap)	Khoảng $[L_i, R_i]$, Quét đường	Bảo toàn khoảng trống giữa các đoạn	Cần nhớ chèn $x + 1$, trọng số đoạn thay đổi	$O(N \log N)$
Nén Lưới 2D	Ma trận $10^9 \times 10^9$, mê cung	Đưa ma trận khổng lồ về kích thước $O(N^2)$	Tốn bộ nhớ hơn 1D ($O(N^2)$ ma trận)	$O(N^2)$
PBDS Ordered Set	Trực tuyến (Online streaming)	Thêm/xóa động, truy vấn thứ hạng $O(\log N)$	Hằng số cây đồ đen lớn hơn mảng phẳng	$O(N \log N)$

7. Cạm bẫy Phòng thi & Sai lầm Thường gặp (Pitfalls)

- Quên xóa phần tử trùng lặp (`std::unique`):**
- Nếu chỉ `std::sort` mà không gọi `.erase(unique(...), end())`, mảng `vals` vẫn chứa các bản sao.
- Khi đó `lower_bound` sẽ trả về chỉ số của bản sao đầu tiên, làm sai lệch kích thước miền giá trị và gây tràn mảng Fenwick Tree / Segment Tree.
- Lỗi chỉ số 0-based vs 1-based khi kết hợp Fenwick Tree (BIT):**
- `lower_bound(...)` - `vals.begin()` trả về chỉ số 0-based (từ 0 đến $K - 1$).
- Tuy nhiên, cây Fenwick Tree (BIT) bắt buộc phải hoạt động trên chỉ số **1-based** (từ 1 đến K).
- Quy tắc khắc phục:* Luôn cộng thêm 1 khi dùng cho BIT:

```
cpp int rank_1_based = (lower_bound(vals.begin(), vals.end(), x) - vals.begin()) + 1;
```
- Tràn số long long khi tính diện tích / chiều dài gốc:**
- Hiệu `vals[i + 1] - vals[i]` có thể lên tới $2 \cdot 10^9$, tích lũy diện tích 2D có thể vượt quá $2 \cdot 10^9$. Bắt buộc khai báo kiểu `long long` cho các biến diện tích/tổng độ dài.
- Quên chèn biên thế giới vào lưới 2D:**
- Khi nén lưới 2D, nếu không chèn thêm các tọa độ biên 0, MAX_COORD, đường đi ngoài viền mê cung có thể bị cắt đứt.

8. Tài liệu Tham khảo & Đọc thêm

- USACO Guide (Silver Section):**
- [Coordinate Compression](#) — Hướng dẫn toàn diện nén điểm, nén đoạn và ứng dụng.
- Competitive Programmer's Handbook** — *Antti Laaksonen*:
- Chapter 9: Range Queries (Coordinate Compression with Fenwick Tree).
- VNOI Wiki:**
- [Kỹ thuật Rời rạc hóa](#).
- Codeforces Edu Course:**



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.